

Investigación profunda: diseño de un CRM tipo Salesforce sin depender de Google Sheets

Potter, la idea que planteaste **va por un muy buen camino**, pero después de contrastarla con la arquitectura de Salesforce Service Cloud, PostgreSQL, Flask, Railway y buenas prácticas de seguridad, haría varios cambios antes de empezar a programar.

Lo más importante es definir correctamente el producto: por lo que describís, no necesitás replicar todo Salesforce. Lo que estás diseñando se parece mucho más a un **CRM operativo de atención y gestión de casos**, equivalente conceptualmente a una parte de **Salesforce Service Cloud**: casos, colas, agentes, asignaciones, estados, auditoría, calidad, búsqueda y reportes. Salesforce utiliza precisamente conceptos como Case Queues, reglas de asignación y reglas de escalamiento para este tipo de operación. ¹

Tu documento ya identifica correctamente los grandes módulos: roles jerárquicos, búsqueda global, reportes, calidad TYQ, vista 360° del caso y una interfaz inspirada en Lightning. [filecite0turn0file0](#)

Mi recomendación sería construirlo como una plataforma propia cuya **fuentes oficial de información sea PostgreSQL**, dejando Google Sheets completamente fuera de la operación diaria.

Evaluación de tu arquitectura actual

La base es correcta:

Flask + PostgreSQL + Tailwind + Railway es perfectamente razonable para este proyecto. Flask recomienda estructurar aplicaciones grandes mediante *application factories* y Blueprints, que permiten separar módulos como administración, casos, calidad y reportes. Para producción, Flask recomienda ejecutar la aplicación con un servidor WSGI dedicado como Gunicorn en lugar del servidor de desarrollo. ²

Pero cambiaría cinco cosas importantes de tu propuesta.

Tu propuesta	Mi recomendación
Flask + API REST interna	Flask modular + capa de servicios; API REST solo donde haga falta
<code>sub_rols</code> en JSONB	Crear tablas <code>queues</code> y <code>queue_memberships</code>
<code>Cases</code> con prácticamente toda la información	Separar cliente, pedido, caso, historial, asignaciones y auditorías
Búsqueda con muchos <code>ILIKE</code>	B-tree + <code>pg_trgm</code> + Full Text Search
Pandas generando todos los reportes dentro de Flask	Reportes pequeños directos; reportes grandes mediante worker

Tu propuesta	Mi recomendación
Archivos dentro del servidor	Object Storage / Railway Bucket
Roles solamente	Roles + permisos + autorización del objeto
Estados con ENUM rígido	Tablas configurables para estados/tipos cuando sea necesario
Railway "autoescala"	Diseñar aplicación stateless y escalar por réplicas cuando sea necesario

Hay otro punto importante: **Railway no aumenta automáticamente el número de réplicas de tu aplicación según la carga tal como está escrito en la propuesta.** Railway permite escalamiento horizontal configurando réplicas y realiza escalado vertical de recursos; las réplicas horizontales se configuran explícitamente. Además, no existe afinidad de sesión entre réplicas, por lo que cualquier arquitectura preparada para escalar debe mantener su estado fuera del proceso web. ³

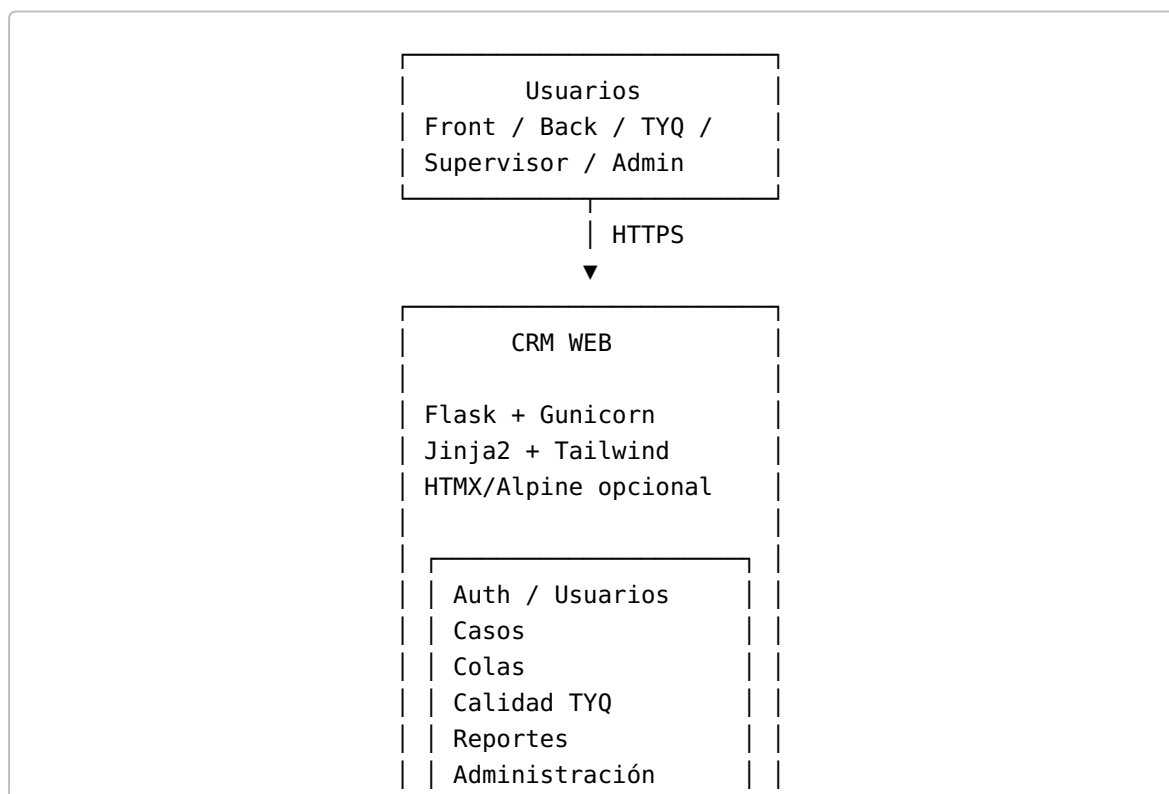
Esto no es un problema para tu CRM. Simplemente conviene diseñarlo bien desde el comienzo.

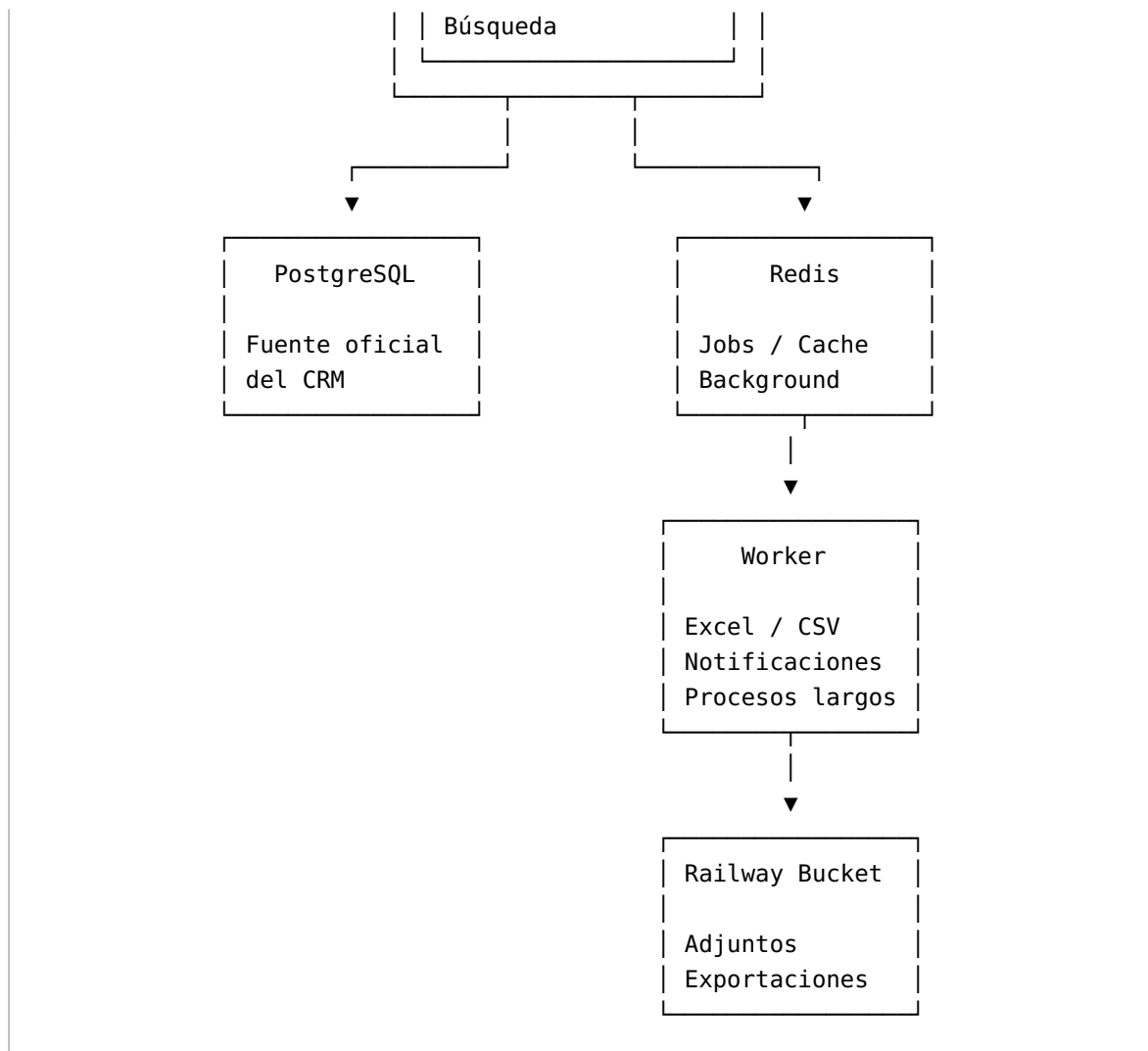
Arquitectura que construiría

Yo utilizaría un **monolito modular**, no microservicios.

Para una primera versión empresarial del sistema, los microservicios agregarían complejidad sin aportarte demasiado. Flask permite separar perfectamente los dominios mediante Blueprints y una fábrica de aplicación. ⁴

La arquitectura objetivo sería aproximadamente esta:





Railway actualmente ofrece PostgreSQL, Redis, almacenamiento de objetos compatible con S3 y opciones de alta disponibilidad para PostgreSQL y Redis, por lo que esta arquitectura puede mantenerse completamente dentro de Railway inicialmente. ⁵

El frontend

Mantendría:

Jinja2 + Tailwind CSS.

Y agregaría una pequeña capa de JavaScript para funcionalidades como:

- búsqueda instantánea;
- filtros;
- modales;
- actualización de estados;
- autocompletado;
- notificaciones;
- cambios parciales de pantalla.

No empezaría con React.

Para un CRM interno donde las pantallas son principalmente tablas, formularios, paneles y vistas de registro, el renderizado del servidor mantiene considerablemente más simple la aplicación.

La vista del caso debería inspirarse particularmente en el modelo de Salesforce de **Record Page + Path**. Salesforce utiliza un Path horizontal basado en estados y vistas de registro que pueden separar la información en varias columnas. ⁶

El backend

Organizaría el proyecto aproximadamente así:

```
crm/
├──
├── app/
│   ├── __init__.py
│   ├── extensions.py
│   ├──
│   ├── auth/
│   ├── cases/
│   ├── customers/
│   ├── orders/
│   ├── queues/
│   ├── quality/
│   ├── reports/
│   ├── search/
│   ├── notifications/
│   ├── admin/
│   ├──
│   ├── services/
│   ├── repositories/
│   ├── permissions/
│   ├── tasks/
│   ├── templates/
│   └── static/
├──
├── migrations/
├── tests/
├── wsgi.py
├── requirements.txt
└── railway.json
```

Usaría SQLAlchemy para las transacciones y Alembic para controlar las modificaciones de la base de datos. Alembic mantiene scripts versionados de actualización de esquema y permite ejecutar migraciones secuenciales hasta la revisión actual, algo fundamental cuando el CRM ya está en producción. ⁷

No crearía una API REST interna completa todavía

Acá cambiaría directamente tu propuesta.

Si una página Jinja necesita crear un caso, no haría:

```
Jinja
  ↓
REST API
  ↓
Business Logic
  ↓
PostgreSQL
```

Haría:

```
Route Flask
  ↓
CaseService
  ↓
Repository / SQLAlchemy
  ↓
PostgreSQL
```

Y más adelante:

```
/api/v1/cases
  ↓
CaseService
  ↓
PostgreSQL
```

De esta manera la lógica de negocio está centralizada, pero no mantenés innecesariamente dos interfaces diferentes desde el primer día.

La API sí tendría mucho sentido posteriormente para integraciones SAP, Wise, Salesforce, aplicaciones móviles o automatizaciones externas.

Modelo de datos recomendado

Esta es probablemente la parte que más modificaría.

Tu tabla `Cases` actualmente está intentando resolver casi todo el sistema.

```
Cases
├ pedido
├ SKU
├ tienda
├ dirección
├ SAP
```

```
├ agente front
├ agente back
├ observaciones
└ estado
```

Eso puede funcionar inicialmente, pero después aparecen problemas.

Por ejemplo:

¿Qué ocurre si un pedido tiene tres casos distintos?

¿Qué ocurre si el caso cambia cuatro veces de agente?

¿Cómo sé quién cambió el estado?

¿Cómo sé qué valor tenía un SAP antes de modificarlo?

¿Cómo agrego comentarios sin modificar `observaciones` ?

¿Cómo tengo varias auditorías TYQ?

¿Cómo agrego archivos?

Para un CRM estilo Service Cloud, recomiendo separar estas responsabilidades.

Usuarios, roles y colas

En vez de:

```
User
sub_rols JSONB
```

utilizaría:

```
users
roles
queues
queue_memberships
```

Conceptualmente:

```
USERS
id
name
email
password_hash
```

```
role_id
active
created_at
last_login_at
```

ROLES

```
id
code
name
```

QUEUES

```
id
code
name
description
active
```

QUEUE_MEMBERSHIPS

```
id
queue_id
user_id
created_at
created_by
```

Entonces:

```
Jorge
|
|—— Solicitudes BGH
|
|—— Reembolsos
|
|—— Cambio de dirección
```

se convierte en relaciones de base de datos reales.

Salesforce sigue precisamente el concepto de **Case Queue**, donde casos pueden ser enviados a una cola y posteriormente gestionados por los usuarios correspondientes; también permite utilizar reglas para asignarlos automáticamente. ¹

Yo evitaría usar JSONB para relaciones como permisos o membresías. Guardaría JSONB para información que realmente sea flexible. PostgreSQL soporta consultas e índices GIN sobre JSONB, por lo que es excelente para datos dinámicos, pero las relaciones fundamentales de negocio deberían seguir siendo relacionales. ⁸

Cliente y pedido

También separaría al cliente y al pedido del caso:

```
customers
-----
id
name
document_type
document_number
email
phone

orders
-----
id
order_number
customer_id
store
tracking
created_at
```

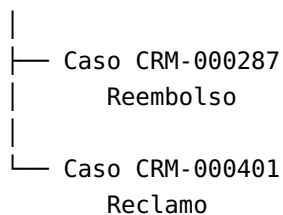
Y después:

```
cases
-----
id
case_number
order_id
customer_id
case_type_id
status_id
priority
queue_id
assigned_to
created_by
created_at
updated_at
resolved_at
closed_at
```

Así un mismo pedido puede tener:

```
Pedido 125873

├─ Caso CRM-000134
|   Cancelación
```



Eso se parece mucho más al concepto real de CRM.

Tipos de caso

Acá resolvería el antiguo concepto de “hojas”.

En lugar de:

Hoja Cancelaciones
Hoja Reembolso
Hoja Solicitudes
Hoja Dirección

tendrías:

case_types

Por ejemplo:

Código	Nombre
CANCEL	Cancelación
REFUND	Reembolso
ADDRESS	Cambio de dirección
REQUEST	Solicitudes BGH
DELIVERY	Problemas de entrega

Incluso podrías definir:

case_type_fields

para indicar qué campos necesita cada formulario.

Eso permitiría que Administración controle algunos formularios sin cambiar código.

Campos específicos

No intentaría convertir absolutamente todas las columnas históricas de Sheets en columnas de `cases`.

Haría un modelo híbrido.

Campos importantes y buscados frecuentemente:

```
order_number
dni
tracking
sap_zre
sap_zech
salesforce_case
email
phone
status
assigned_to
```

→ columnas reales.

Información específica de cada tipo:

```
{
  "motivo_cancelacion": "cliente solicito",
  "producto_retirado": true,
  "fecha_prometida": "2026-09-20"
}
```

→ `custom_data JSONB`.

PostgreSQL permite indexar JSONB mediante GIN si posteriormente determinado dato dinámico necesita consultarse frecuentemente. ⁸

Historial de actividad

Esta tabla es indispensable:

```
case_events
-----
id
case_id
user_id
event_type
field_name
old_value
```

```
new_value
comment
created_at
```

Ejemplo:

```
09:32  Jorge creó el caso
09:35  Sistema → Cola Cancelaciones
09:48  María tomó el caso
10:02  María cambió Nuevo → En Proceso
10:17  María agregó SAP ZRE 928193
10:21  María cambió En Proceso → Resuelto
13:40  Laura TYQ auditó el caso: 94%
```

Ese sería el panel lateral derecho de la vista 360°.

Auditoría TYQ

Tu idea de crear `Quality_Audits` separada es mejor que incorporar directamente los campos de calidad dentro de `Cases`.

La ampliaría:

```
quality_audits
-----
id
case_id
auditor_id
agent_id
score
feedback
created_at

quality_audit_items
-----
id
audit_id
criterion_id
answer
score
comment

quality_criteria
-----
id
name
description
```

weight
active

Esto permite que el checklist cambie con el tiempo sin modificar todos los casos anteriores.

Y además permitiría más adelante:

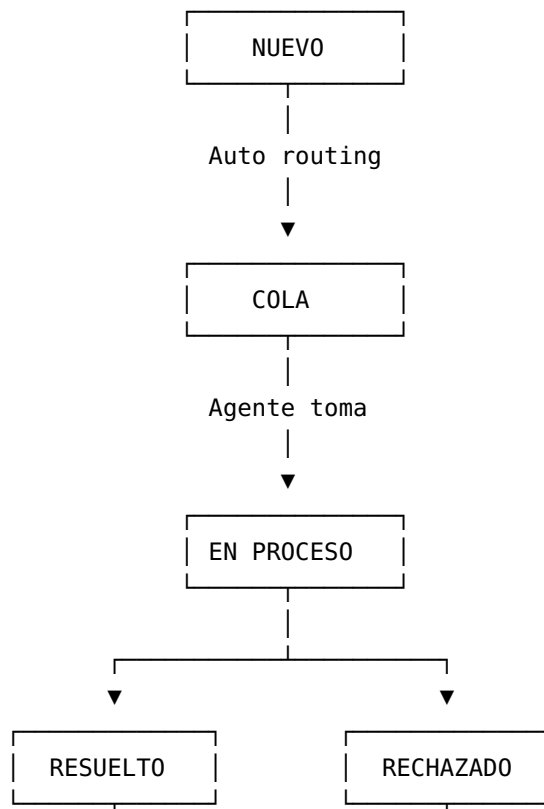
promedio por agente
promedio por supervisor
promedio por cola
promedio por tipo de caso
errores frecuentes
evolución mensual

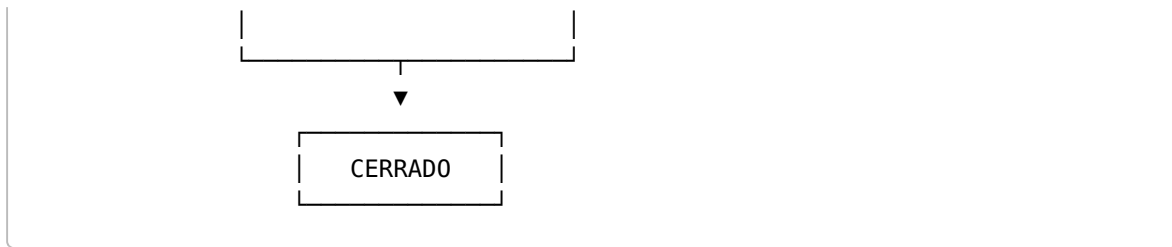
Flujo operativo estilo Salesforce

Esta es la funcionalidad que realmente hará que el sistema se sienta como Salesforce. No el color azul o los chevrons.

Salesforce Service Cloud combina casos, colas, reglas de asignación y escalamiento. Por ejemplo, una regla puede mandar automáticamente determinados casos a una cola y otra puede reasignarlos cuando permanecen demasiado tiempo sin resolver. ¹

Yo implementaría el ciclo:





El Path horizontal de Salesforce precisamente representa etapas utilizando chevrons, por lo que tu idea visual está muy alineada con ese comportamiento. ⁹

Pero agregaría tres motores.

Enrutamiento automático

Ejemplo:

```
SI case_type = "Reembolso"  
Y tienda = "BGH"  
→ Cola Reembolsos BGH
```

Otro:

```
SI case_type = "Cancelación"  
→ Cola Cancelaciones
```

Y después:

```
SI prioridad = Alta  
→ Queue Priority = Alta
```

No asignaría necesariamente un caso directamente a una persona.

Primero:

```
Caso → Cola
```

Después:

```
Cola → Agente
```

Es un modelo mucho más flexible.

SLA y escalamiento

Ejemplo:

```
Caso creado
  ↓
2 horas
  ↓
¿Sigue Nuevo?
  ↓
Supervisor recibe alerta
  ↓
4 horas
  ↓
Escalamiento
```

Las reglas de escalamiento forman parte del modelo de gestión de casos de Salesforce y permiten actuar de acuerdo con la antigüedad y estado del caso. ¹⁰

No es indispensable para el primer MVP, pero dejaría la arquitectura preparada.

Auditoría de cambios

Todo cambio importante debería producir un evento:

```
CREATE_CASE
ASSIGN_CASE
CHANGE_STATUS
EDIT_FIELD
ADD_NOTE
ADD_ATTACHMENT
QUALITY_AUDIT
REOPEN_CASE
CLOSE_CASE
```

Nunca dependería solamente de:

```
cases.updated_at
```

porque eso te dice cuándo cambió el caso, pero no **qué, quién ni por qué**.

Búsqueda, reportes y experiencia de usuario

Acá encontré otro punto que vale la pena corregir de tu propuesta.

El `ILIKE` solo no es suficiente

Esto:

```
Case.query.filter(  
    or_(  
        Case.pedido_id.ilike(f"%{query}%"),  
        Case.sap_zre.ilike(f"%{query}%"),  
        Case.sap_zech.ilike(f"%{query}%")  
    )  
)
```

funciona.

Pero no diseñaría la búsqueda global final de esa manera.

PostgreSQL proporciona `pg_trgm`, que permite crear índices GIN/GiST capaces de acelerar búsquedas mediante `LIKE`, `ILIKE` y similitud textual incluso cuando el término no está anclado al comienzo del campo. Para búsqueda de texto completo, PostgreSQL también proporciona `tsvector` / `tsquery`, y su documentación recomienda normalmente GIN como índice para Full Text Search. ¹¹

Haría tres niveles.

Identificadores exactos: B-tree

```
pedido  
DNI  
tracking  
ZRE  
ZECH  
case_number
```

PostgreSQL recomienda B-tree para datos escalares y escenarios concurrentes. ¹²

Texto parcial: `pg_trgm`

```
nombre  
apellido  
email  
tracking  
SAP
```

Texto largo: Full Text Search

observaciones
notas
feedback TYQ

La barra superior podría entonces buscar:

FRC-1579394052352-01

y devolver:

CASOS
CRM-001248 | Cancelación | FRC-1579394052352-01

PEDIDOS
FRC-1579394052352-01 | Juan Pérez

CLIENTES
Juan Pérez | DNI 4xxxxxxx

Eso sería mucho más cercano a una búsqueda CRM real.

Reportes

Tu enfoque con `pandas + openpyxl + BytesIO` está bien para exportaciones moderadas.

Pero no haría todas las exportaciones dentro de la petición HTTP.

Separaría:

Reporte pequeño
↓
generación inmediata

Reporte grande
↓
crear Job
↓
Worker
↓
generar Excel
↓
Storage Bucket
↓
"Tu reporte está listo"

Las colas de trabajo existen precisamente para mover procesos largos o bloqueantes fuera de los procesos web; RQ y Celery soportan workers dedicados para esta clase de tareas. ¹³

Railway puede provisionar Redis directamente, por lo que podrías utilizar:

```
Flask
↓
Redis
↓
RQ Worker
```

sin agregar una infraestructura complicada. ¹⁴

Para este proyecto elegiría **RQ antes que Celery**. No porque Celery sea peor, sino porque tu caso todavía no necesita toda su complejidad.

Adjuntos

También agregaría desde el principio:

```
attachments
-----
id
case_id
uploaded_by
filename
storage_key
mime_type
size
created_at
```

Los archivos no deberían guardarse dentro de PostgreSQL ni dentro del contenedor Flask.

Railway ofrece Buckets privados compatibles con S3 y permite entregar archivos mediante URLs firmadas o a través del backend; la propia documentación recomienda este patrón para archivos subidos por usuarios. ¹⁵

Entonces podrías adjuntar:

```
factura.pdf
captura.png
comprobante.pdf
respuesta_sap.xlsx
```

al caso.

Seguridad, permisos y despliegue

Tu RBAC está bien conceptualmente, pero falta una distinción crucial:

ROL

no es lo mismo que:

AUTORIZACIÓN SOBRE UN CASO

Un `agente_back` puede tener permiso para abrir casos.

Pero:

¿puede abrir **ese** caso?

OWASP recomienda verificar autorización en cada solicitud y verificar específicamente el acceso al objeto solicitado, no simplemente comprobar que el usuario tenga un determinado rol. ¹⁶

Por ejemplo:

```
@cases.route("/cases/<uuid:case_id>")
@login_required
def detail(case_id):

    case = case_service.get(case_id)

    if not can_view_case(current_user, case):
        abort(403)

    return render_template("cases/detail.html", case=case)
```

`can_view_case()` podría verificar:

Admin	→ todos
Supervisor	→ todos / equipo
TYQ	→ casos habilitados para calidad
Agente Front	→ casos creados por él
Agente Back	→ colas asignadas

Contraseñas

En lugar de dejar simplemente:

```
password_hash
```

en la especificación, definiría explícitamente el algoritmo.

OWASP recomienda actualmente Argon2id como primera opción para nuevos sistemas de almacenamiento de contraseñas y desaconseja algoritmos rápidos de hashing genérico para este propósito. ¹⁷

CSRF

Como vas a usar sesiones y formularios Flask, también activaría protección CSRF global.

Flask-WTF proporciona `CSRFProtect`, y OWASP recomienda proteger las operaciones que modifican estado cuando la autenticación depende de cookies. ¹⁸

Auditoría

Tendría una tabla separada:

```
audit_logs
-----
id
user_id
action
entity_type
entity_id
old_data
new_data
ip_address
created_at
```

Especialmente para:

```
cambio de permisos
creación de usuario
reassignación
cambio de estado
modificación SAP
eliminación
exportaciones
auditoría TYQ
```

No permitiría borrar silenciosamente registros importantes.

Preferiría:

```
active = false
deleted_at = ...
```

en entidades donde conservar la trazabilidad sea necesaria.

Railway

En Railway yo tendría al menos:

```
CRM Web
PostgreSQL
```

y después agregaría:

```
Redis
Worker
Bucket
```

cuando aparezcan reportes pesados, notificaciones o adjuntos.

Para producción configuraría:

```
/health
```

Railway utiliza healthchecks para verificar un nuevo deployment antes de dirigirle tráfico; sin embargo, esos healthchecks verifican el despliegue y no funcionan como monitorización continua una vez que la versión ya está en producción. ¹⁹

También activaría backups desde el inicio.

Railway ofrece snapshots de volúmenes, Point-in-Time Recovery para PostgreSQL y backups lógicos con `pg_dump`; su propia guía recomienda combinar estas capas para producción y realizar pruebas reales de restauración. ²⁰

Eso es particularmente importante porque después de la migración:

PostgreSQL será la única fuente de verdad.

Ya no existirá Google Sheets como respaldo improvisado.

Roadmap que considero más seguro

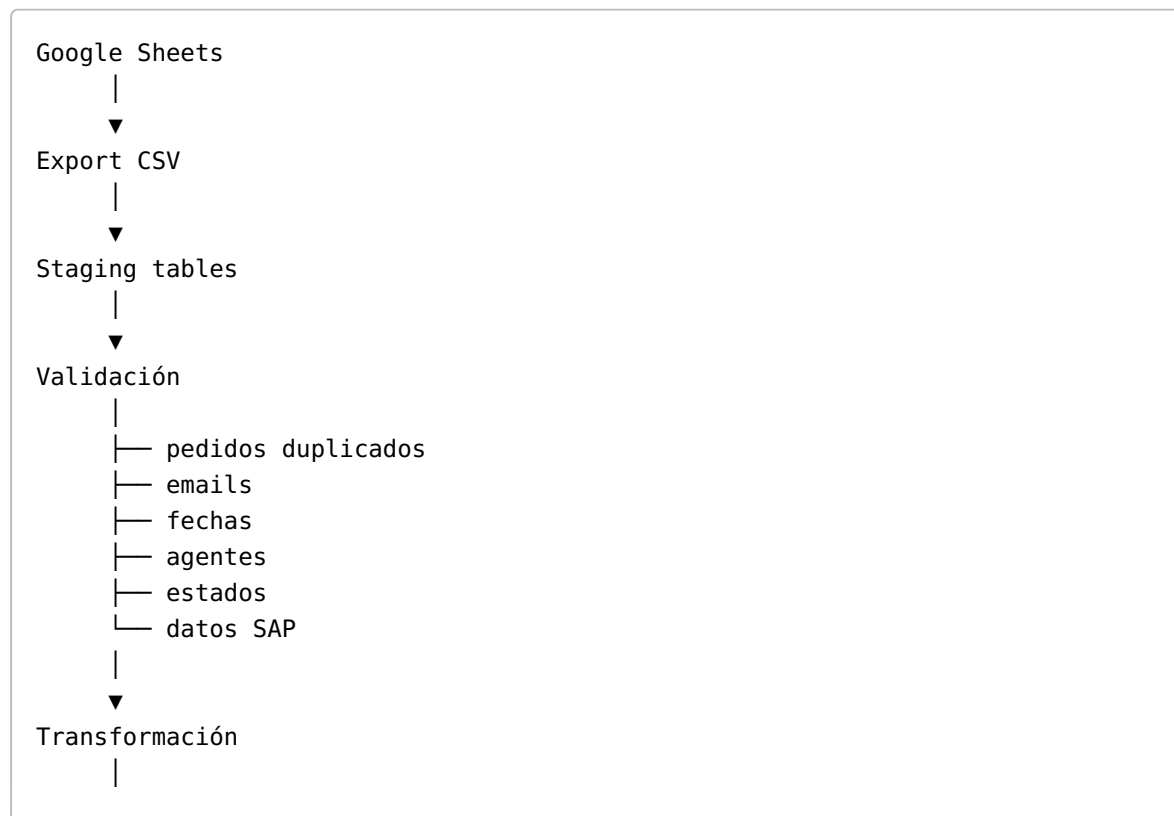
Yo modificaría un poco tus cinco semanas.

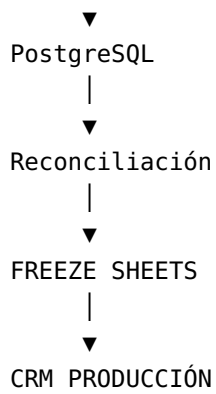
Para una sola persona, cinco semanas me parece razonable para un **MVP funcional**, pero bastante agresivo para considerar el CRM terminado a nivel producción.

Haría primero una versión muy sólida del flujo central.

Etapas	Resultado
Base	Flask, PostgreSQL, Railway, migraciones y estructura modular
Seguridad	Login, usuarios, roles, permisos y sesiones
Core CRM	Clientes, pedidos, casos, tipos y estados
Operación	Colas, asignaciones y panel Back Office
Trazabilidad	Timeline, comentarios e historial
Front	Nuevo caso y validaciones
Búsqueda	Pedido, DNI, tracking, SAP y cliente
Calidad	Auditorías y dashboard TYQ
Reporting	Filtros, CSV y Excel
Migración	Importación histórica de Sheets
Hardening	Backups, tests, logs y permisos
Go-live	PostgreSQL como fuente única

El punto crítico sería hacer la migración así:





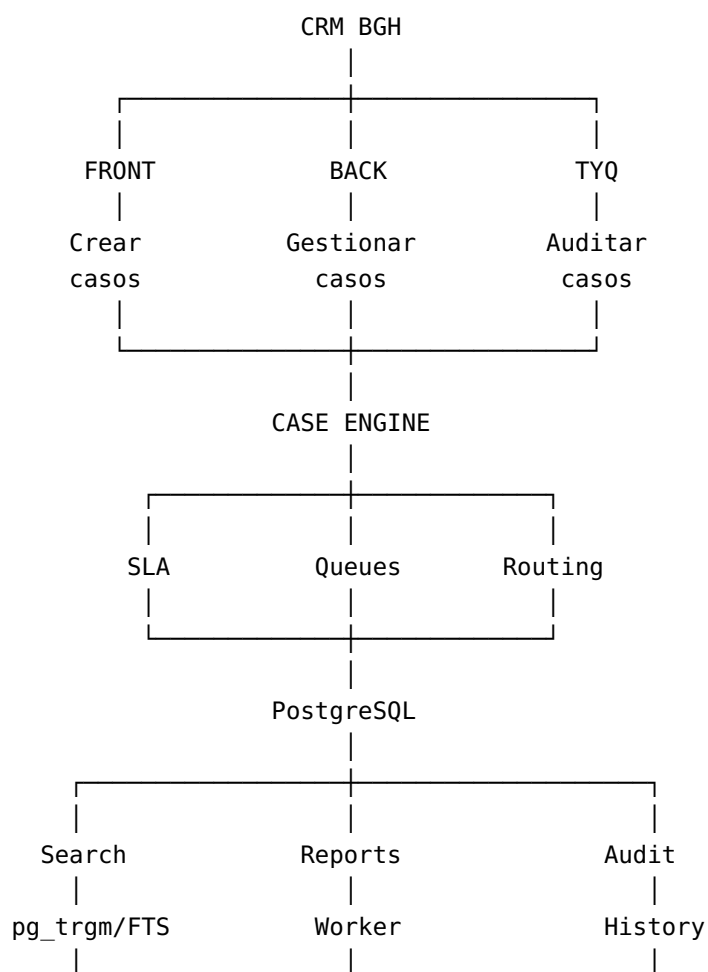
No mantendría una sincronización permanente:

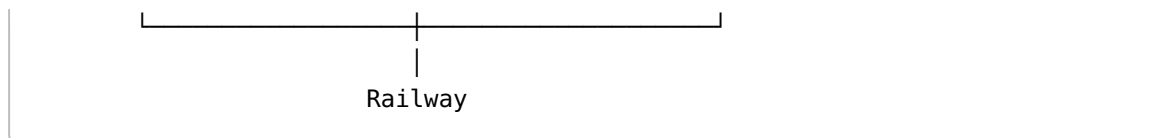
CRM ↔ Google Sheets

porque volverías a tener dos fuentes potencialmente contradictorias.

Google Sheets debería convertirse simplemente en **fuentes histórica de migración**.

La base final que yo construiría sería:





Mi stack final recomendado quedaría:

Componente	Tecnología
Backend	Python + Flask
Arquitectura	Monolito modular
Templates	Jinja2
UI	Tailwind CSS
Interactividad	HTMX/Alpine opcional
ORM	SQLAlchemy
Migraciones	Alembic
Base de datos	PostgreSQL
Search	B-tree + pg_trgm + PostgreSQL FTS
Auth	Flask-Login + Argon2id
Formularios	Flask-WTF + CSRF
Background jobs	RQ + Redis
Excel	openpyxl
CSV	streaming / generación directa
Archivos	Railway Storage Bucket
Servidor	Gunicorn
Hosting	Railway
CI/CD	GitHub → Railway
Backup	Railway backups + PITR + pg_dump
Arquitectura futura	API <code>/api/v1</code> para integraciones

La idea central sería esta:

No hacer una copia visual de Salesforce. Hacer un CRM propio que copie las buenas ideas operativas de Salesforce: registros centralizados, colas, reglas, permisos, vista 360°, historial completo, búsqueda global, automatización, métricas y calidad.

Eso transforma el proyecto que planteaste de un “reemplazo de Google Sheets con Flask” en un **verdadero sistema de gestión de casos**, preparado para que más adelante puedas agregar SLA, notificaciones, SAP, automatizaciones, dashboards, APIs y otras integraciones sin reconstruir la base del sistema. Salesforce mismo basa buena parte de su experiencia de Service Cloud en la combinación de

registros, vistas compactas, Path, colas, reglas de asignación y escalamiento, por lo que adoptar esos conceptos —sin intentar clonar toda la plataforma— es una dirección técnicamente sólida para tu caso.

21

-
- 1 Enhance Customer Service with Automated Case Management
https://trailhead.salesforce.com/fr/content/learn/modules/service_lex/service_lex_case_manage?utm_source=chatgpt.com
 - 2 4 Application Factories — Flask Documentation (3.1.x)
https://flask.palletsprojects.com/en/stable/patterns/appfactories/?utm_source=chatgpt.com
 - 3 Optimize Performance | Railway Docs
https://docs.railway.com/deployments/optimize-performance?utm_source=chatgpt.com
 - 5 Databases | Railway Docs
https://docs.railway.com/databases?utm_source=chatgpt.com
 - 6 9 21 Path (Beta) | Legacy Components | Lightning Component Reference | Salesforce Developers
https://developer.salesforce.com/docs/platform/lightning-component-reference/guide/aura-path.html?utm_source=chatgpt.com
 - 7 Tutorial — Alembic 1.20.0 documentation
https://alembic.sqlalchemy.org/en/latest/tutorial.html?utm_source=chatgpt.com
 - 8 PostgreSQL: Documentation: 18: 65.4. GIN Indexes
https://www.postgresql.org/docs/18/gin.html?utm_source=chatgpt.com
 - 10 Enhance Customer Service with Automated Case Management
https://trailhead.salesforce.com/en/content/learn/modules/service_lex/service_lex_case_manage?utm_source=chatgpt.com
 - 11 PostgreSQL: Documentation: 17: F.33. pg_trgm — support for similarity of text using trigram matching
https://www.postgresql.org/docs/17/pgtrgm.html?utm_source=chatgpt.com
 - 12 PostgreSQL: Documentation: 18: 13.7. Locking and Indexes
https://www.postgresql.org/docs/18/locking-indexes.html?utm_source=chatgpt.com
 - 13 RQ: Workers
https://python-rq.org/docs/workers/?utm_source=chatgpt.com
 - 14 Redis | Railway Docs
https://docs.railway.com/databases/redis?utm_source=chatgpt.com
 - 15 Storage Buckets | Railway Docs
https://docs.railway.com/storage-buckets?utm_source=chatgpt.com
 - 16 Authorization - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html?utm_source=chatgpt.com
 - 17 Password Storage - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html?utm_source=chatgpt.com
 - 18 CSRF Protection — Flask-WTF Documentation (1.3.x)
https://flask-wtf.readthedocs.io/en/latest/csrf/?utm_source=chatgpt.com
 - 19 Healthchecks | Railway Docs
https://docs.railway.com/deployments/healthchecks?utm_source=chatgpt.com
 - 20 Point-in-Time Recovery | Railway Docs
https://docs.railway.com/volumes/point-in-time-recovery?utm_source=chatgpt.com